

Design, FPGA Implementation, and ASIC Flow of a Custom 8-Core SIMT GPU for General-Purpose Computing

Project Report

Project Mentors:

Mukul Paliwal (231EE134)

Ratan Y Mallya (231EC146)

Sirigiri Tarun (231EC156)

Team Members:

Rushil Jain (241EC148)

Shamit Hoysal (241EE253)

Vamshikrishna V Bidari (241EC58)

Vikram Singh (241EC164)



NATIONAL INSTITUTE OF TECHNOLOGY KARNATAKA,
SURATHKAL

April 7, 2026

Contents

1	Introduction	3
1.1	Project Statement	3
1.2	Background Survey	3
2	GPU Architecture & Memory Hierarchy	4
2.1	Top-Level Architecture and The Dispatcher	4
2.2	Physical Memory Separation	6
2.3	Compute Core Components	6
3	Custom Instruction Set Architecture & Arithmetic	9
3.1	16-Bit Instruction Format and Decoding	9
3.2	Fixed-Point Arithmetic & The Division Bottleneck	9
3.3	Branching, Control Flow, and Architectural Choices	10
4	SIMT Execution & Multi-Core Scaling	11
4.1	Core Execution Pipeline	11
4.2	Thread-Level Execution and Convergence	12
4.3	The 8-Core Expansion & Clock Tuning	12
4.4	Hardware Optimization: The Dispatcher State Bug	12
5	Physical FPGA Implementation & Peripherals	13
5.1	System Integration & Physical I/O	13
5.2	Asynchronous Dual-Clock VGA Pipeline	13
5.3	Hardware Color Pipeline & Memory Optimization	14
5.4	Memory-Mapped I/O (MMIO) & Hardware Navigation	14
6	Evaluation & Benchmarking: The Mandelbrot Set	15
6.1	Algorithmic Mapping & The Assembly Kernel	15
6.2	High-Resolution 800x600 Hardware Rendering	16
6.3	Hardware Resource Utilization & Synthesis Results	17
6.4	Simulation vs. Real-World Silicon Performance	18
7	ASIC Design Flow: RTL-to-GDSII	19
7.1	Physical Design Configuration	19
7.2	Clock Tree Synthesis (CTS) and Distribution	20
7.3	GDSII Layout Generation	20
8	Challenges & Learnings	23
8.1	Obstacles Faced	23
8.2	Solutions & Insights	23

9	Conclusion & Future Enhancements	23
9.1	Final Takeaways	23
9.2	Future Work	24
10	References	24

1 Introduction

1.1 Project Statement

The primary objective of this project is the design, simulation, and physical implementation of a custom, minimal Single Instruction, Multiple Threads (SIMT) Graphics Processing Unit (GPU) from scratch using SystemVerilog. While modern commercial GPUs have grown into immensely complex, proprietary black boxes, this project aims to demystify the fundamental architecture of hardware accelerators. By building a fully functional General-Purpose GPU (GPGPU) and deploying it as a physical System-on-Chip (SoC) on a Nexys 4 DDR FPGA board, we expose the underlying mechanics of thread dispatching, memory arbitration, and parallel execution.

GitHub Repository: <https://github.com/RushilJ2603/IEEE-Core-Cascade>

Demo Video: <https://tinyurl.com/gpu-demo>

The resulting custom architecture features a highly scalable 8-core compute cluster capable of managing 32 parallel threads in flight synchronously. It is driven by a custom 16-bit Instruction Set Architecture (ISA) and a 32-bit Q8.24 fixed-point Arithmetic Logic Unit (ALU). To interface this mathematical processing engine with the physical world, the system includes a highly optimized, asynchronous dual-clock VGA pipeline driving an 800x600 resolution display, paired with a Memory-Mapped I/O (MMIO) navigation controller to allow real-time hardware interaction without interrupting core GPU memory spaces.

1.2 Background Survey

SIMT Architecture and General-Purpose Computing At the core of modern GPU computing is the SIMT execution model. Unlike traditional sequential Central Processing Units (CPUs) that rely on complex out-of-order execution and deep branch prediction to execute single threads rapidly, SIMT architectures group threads into blocks that execute the exact same instruction synchronously across multiple data streams. Each thread maintains its own register state, such as its local `%blockIdx` and `%threadIdx` variables, which dictates the specific memory address it will process. This paradigm enables massive parallelization while drastically minimizing the overhead of fetching and decoding instructions independently, making it the industry standard for machine learning, vector mathematics, and real-time graphics.

The "Tiny-GPU" Foundation and Project Upgrades This project builds upon the foundational concepts introduced by the open-source "Tiny-GPU" project by Adam Majmudar. The original architecture demonstrated a minimal Verilog GPU implementation primarily focused on simple matrix arithmetic proofs-of-concept using 2 compute

cores. To support high-performance visual computing and scale to an interactive SoC, our team radically redesigned and expanded this base architecture.

Major architectural upgrades implemented in our system include:

- **8-Core Scaling and Clock Tuning:** Expanding the execution engine to 8 parallel compute cores. Due to the massive routing complexity this introduced, clocking wizards were utilized to safely underclock the GPU compute domain to 95 MHz, guaranteeing timing closure and physical hardware stability on the FPGA.
- **32-Bit Arithmetic & Extended ISA:** Shifted to a 32-bit number system, settling specifically on the Q8.24 fixed-point representation for the Mandelbrot implementation. To support this, the custom ISA was upgraded with new hardware instructions including `FIXED_MUL`, `SLL`, `SRL`, and `SRA`.
- **32-Bit Register Expansion:** Expanding internal registers, block IDs, and the Device Control Register (DCR) to 32 bits to prevent integer overflow during the massive thread dispatches required for high-resolution computing (up to 480,000 threads per frame).
- **Algorithmic Hardware Pivots:** Completely replacing expensive hardware division operations (`DIV`) with reciprocal `FIXED_MUL` optimizations, bypassing severe timing violations while maintaining arithmetic precision.
- **Memory Separation:** Inferring Program Memory and Configuration Memory using distributed LUTs, allowing the entirety of the FPGA’s physical Dual-Port BRAM to be dedicated exclusively as an asynchronous Framebuffer.

The Mandelbrot Set Benchmark While the hardware was designed strictly for general-purpose parallel workloads, we selected the Mandelbrot set rendering algorithm as our primary benchmark application. Fractals serve as the ultimate stress test for parallel GPUs due to their heavy data parallelism, high arithmetic intensity, and natural thread divergence, different pixels escape the fractal bounds at vastly different iteration depths, perfectly testing the asynchronous load/store wait-states of our scheduler.

By pushing our custom 8-core GPU to calculate the Mandelbrot set across 480,000 parallel threads (targeting an 800x600 display), storing the resulting 8-bit iterations in the BRAM, and pushing the data through a custom hardware color mapper to physical VGA pins, we successfully validate the performance, asynchronous memory synchronization, and heavy arithmetic capabilities of our custom silicon design.

2 GPU Architecture & Memory Hierarchy

2.1 Top-Level Architecture and The Dispatcher

The top-level architecture of the GPU is designed to decouple the host interface from the parallel execution engines. The central point of communication between the external

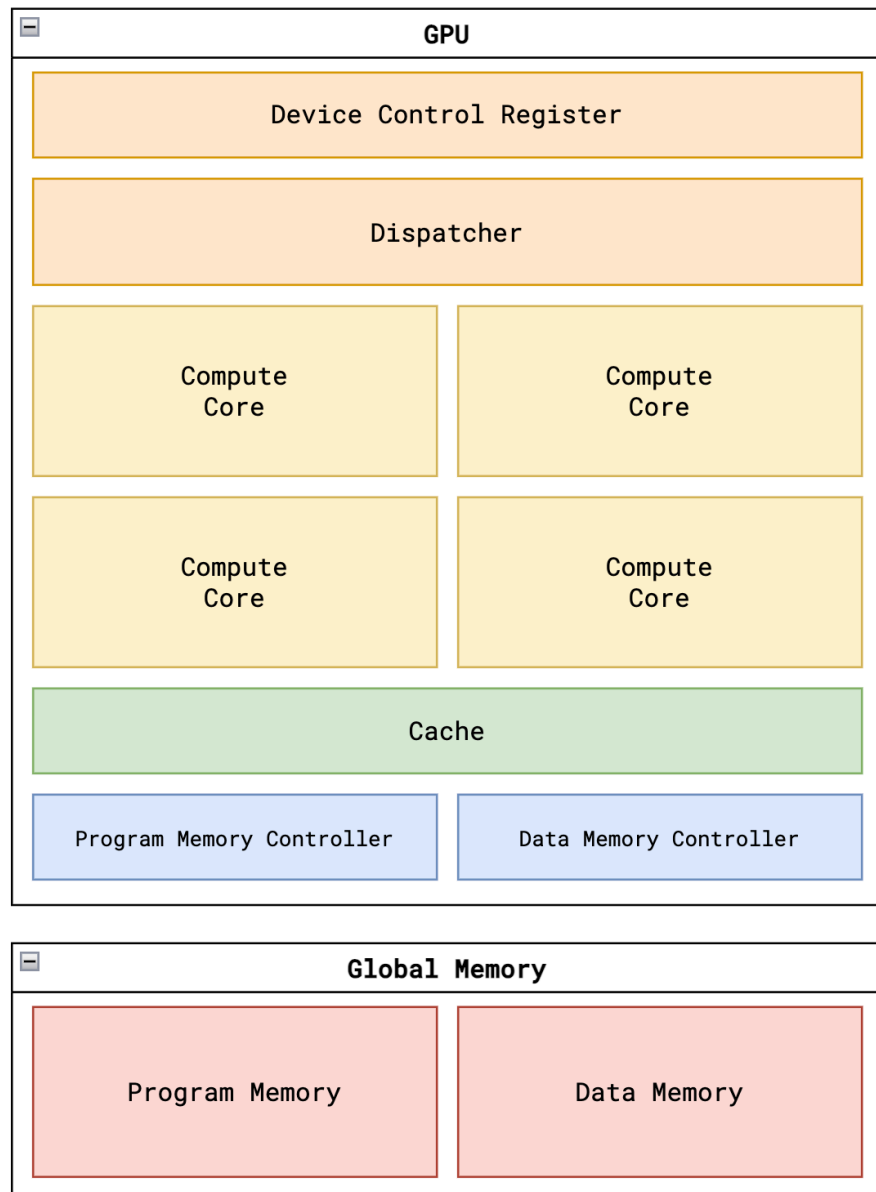


Figure 1: Block diagram integrating the GPU Cores, Dispatcher, Device Control Register, Controllers, and Memory.

hardware (or host processor) and the GPU is the Device Control Register (DCR). In our implementation, the DCR has been expanded to a 32-bit register to store the global `thread_count` metadata. This massive expansion is critical; rendering an 800x600 display requires 480,000 threads, a number that would cause severe integer overflow in standard 8-bit or 16-bit registers.

When the GPU receives a `start` signal, the DCR passes the `thread_count` to the hardware Dispatcher. The Dispatcher calculates the total number of blocks required by dividing the total threads by the `THREADS_PER_BLOCK` parameter (configured to 4 threads per block). Operating as a finite state machine, the Dispatcher constantly monitors the `core_done` signals of the 8 compute cores. As soon as a core finishes its workload, the Dispatcher resets it and immediately assigns it a new 32-bit `block_id`, ensuring that all 8 cores (32 total threads) are constantly saturated with work until the entire frame is processed.

2.2 Physical Memory Separation

A critical architectural pivot made during the hardware implementation phase was the physical separation of memory domains. Typical integrated systems utilize a unified memory architecture where instructions, constants, and output data share the same data bus. However, in a graphics rendering pipeline, bandwidth is the primary bottleneck.

To support real-time 800x600 video outputs without violent data bus contention, we explicitly partitioned the memory space:

- **Program and Configuration Memory:** The 16-bit instructions (the compiled kernel assembly) and the 32-bit global configuration constants (such as coordinates, zoom parameters, and reciprocal math values) are inferred directly using the FPGA’s distributed Look-Up Tables (LUTs).
- **The Framebuffer:** By moving the program and constants to LUTs, the entirety of the FPGA’s physical Dual-Port Block RAM (BRAM) is dedicated exclusively to the Framebuffer. The BRAM was parametrized to a 19-bit address width, providing a depth of 524,288 addresses, more than enough to house the 480,000 pixels required for an 800x600 display.

2.3 Compute Core Components

The GPU relies on 8 parallel compute cores. Each core processes a single block of 4 threads synchronously. The core operates using a classic pipelined approach, divided into specialized hardware sub-modules:

Register Files and SIMT Support The defining feature of a SIMT architecture is its ability to execute the same instruction across multiple data streams without branching the instruction fetcher. To achieve this, each of the 4 threads inside a core contains its

own dedicated 32-bit register file consisting of 16 registers. Registers R0 through R12 are general-purpose registers used for arithmetic and memory operations. However, registers R13, R14, and R15 are hardwired as read-only registers containing the thread's local execution context:

- R13 (%blockIdx): The 32-bit ID of the current block, passed down from the Dispatcher.
- R14 (%blockDim): The number of threads in the block (statically set to 4).
- R15 (%threadIdx): The local ID of the thread within the block (0 to 3).

Register(s)	Variable Mapping & Description
R0	global_tid (Thread ID)
R1, R2	X and Y pixel coordinates
R4, R5	Cx and Cy (Scaled coordinates in the complex plane)
R6, R7	Zx and Zy (The iterative complex variables)
R8	iter_count
R13, R14, R15	Read-only hardware registers (block_id, threads_per_block, local_tid)

Table 1: Register Map Cheat Sheet utilized for the Mandelbrot kernel, detailing the assignment of general-purpose calculation registers and hardwired SIMT thread indices.

By utilizing these read-only registers, a single assembly instruction can force each thread to calculate a unique global memory address, enabling true SIMT parallelism.

The Fetcher and Decoder Each core features a single Fetcher module. Because all 4 threads in a block execute synchronously, the Fetcher only needs to retrieve the 16-bit instruction at the current Program Counter (PC) from the LUT-based program memory once per cycle.

Once fetched, the Decoder acts as a massive combinational logic block. It translates the 16-bit opcode and operands into hardware control signals. For example, it extracts the destination register (Rd), sets the `reg_write_enable` flag, and configures the multiplexers to route data either from the ALU or the Load/Store Unit depending on the instruction.

ALU & Load/Store Unit (LSU) The Arithmetic Logic Unit (ALU) performs all mathematical operations. Upgraded significantly from standard integer ALUs, our design processes 32-bit Q8.24 fixed-point numbers to maintain extreme precision during deep fractal zooms.

The Load/Store Unit (LSU) handles all asynchronous memory requests to the global BRAM Framebuffer. Because memory writes can take multiple clock cycles depending on bus availability, the LSU outputs a 2-bit state signal (`REQUESTING`, `WAITING`, or `DONE`).

The core's scheduler safely halts the execution pipeline until all 4 LSUs in the block report a **DONE** state, preventing data corruption.

Program Counters (PC) and Branching Every thread maintains an independent 8-bit Program Counter. By default, the PC increments by 1 after every instruction. Complex control flow is managed via the **CMP** (Compare) instruction, which evaluates two registers and writes a 3-bit flag into the PC's local **NZP** (Negative, Zero, Positive) register. The **BRnzp** (Branch) instruction subsequently evaluates this flag; if the condition matches, the PC multiplexer routes an immediate address to the PC, executing a jump.

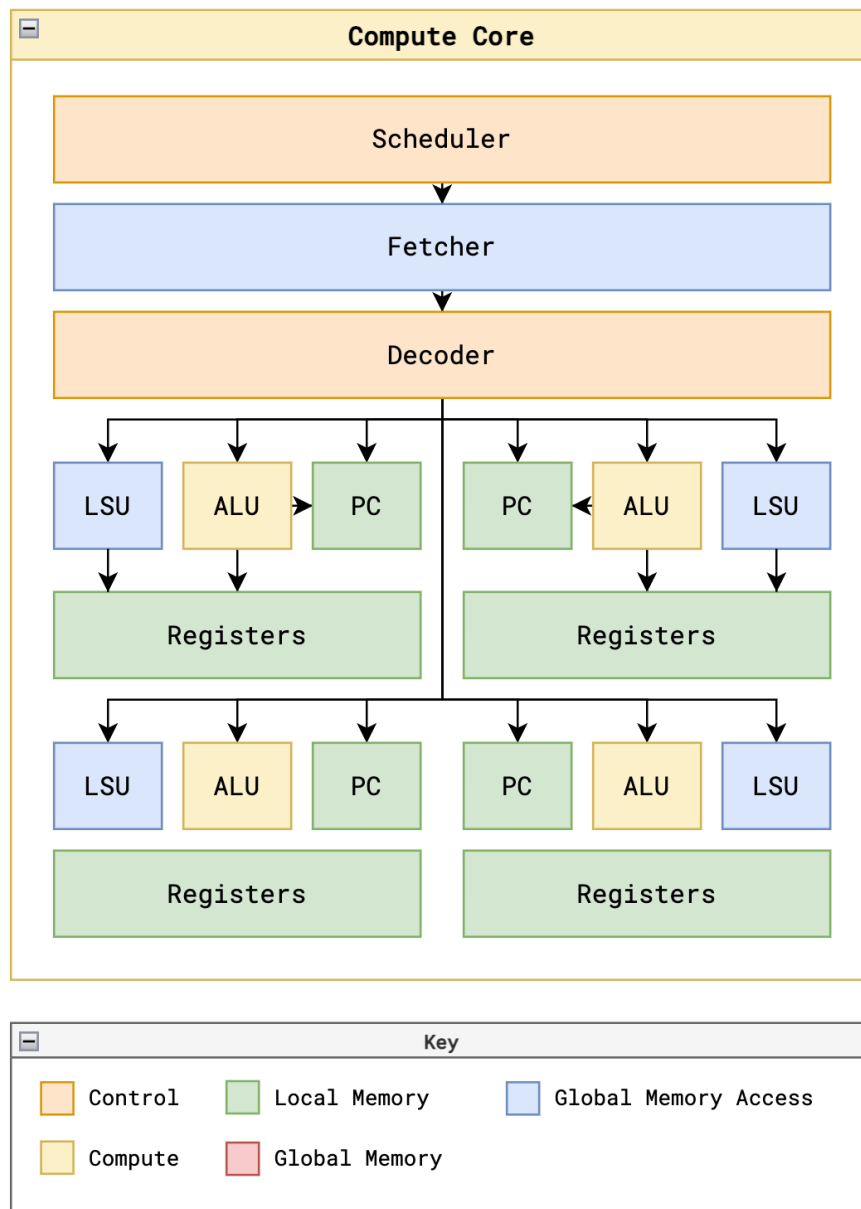


Figure 2: Datapath of a single Compute Core, highlighting the Fetcher, Decoder, ALU, LSU, PC and the 4 independent Thread Register Files.

3 Custom Instruction Set Architecture & Arithmetic

3.1 16-Bit Instruction Format and Decoding

To minimize the memory footprint of the program and reduce the complexity of the hardware decoder, the GPU operates on a custom, highly compact 16-bit instruction format. The 16 bits of each instruction are mapped strictly as follows:

- [15:12] **Opcode**: A 4-bit identifier dictating the operation.
- [11:8] **Destination Register (Rd)**: A 4-bit identifier for the target register.
- [7:4] **Source Register 1 (Rs)**: A 4-bit identifier for the first operand.
- [3:0] **Source Register 2 (Rt) / Immediate**: A 4-bit identifier for the second operand, or a 4-bit immediate value depending on the opcode.

This minimal configuration deliberately imposes strict architectural constraints. With a 4-bit opcode, the ISA can support a maximum of 16 unique instructions. The decoder currently implements exactly 15 instructions, utilizing hardware shifts and custom fractional multiplication to support complex fractal rendering.

Opcode (Hex)	Mnemonic	Description
0000	NOP	No Operation
0001	BRnzp	Branch absolute based on NZP flags
0010	CMP	Compare Rs and Rt, set NZP flags
0011	ADD	Add registers: $Rd = Rs + Rt$
0100	SUB	Subtract registers: $Rd = Rs - Rt$
0101	MUL	Standard Integer Multiply
0110	DIV	Dummy Operation (Outputs 0xDEADBEEF)
0111	LDR	Load from Data Memory into Rd
1000	STR	Store Rd into Data Memory
1001	CONST	Load constant into Rd
1010	FIXED_MUL	Fixed-Point Multiply ($Q8.24 \times Q8.24$)
1011	SLL	Shift Left Logical
1100	SRL	Shift Right Logical
1101	SRA	Shift Right Arithmetic
1111	RET	End thread execution

Table 2: The 15 Custom 16-bit ISA instructions implemented in the GPU Decoder.

3.2 Fixed-Point Arithmetic & The Division Bottleneck

Floating-point mathematics compliant with the IEEE 754 standard requires massive silicon resources, quickly exhausting the physical DSP slices of an FPGA. To circumvent this, the GPU’s Arithmetic Logic Unit (ALU) operates entirely on 32-bit **Q8.24 fixed-point numbers** (8 bits for the integer component, 24 bits for the fractional component).

While standard addition and subtraction work natively with fixed-point formats, multiplication requires specialized hardware. A standard `MUL` instruction is sufficient when multiplying an integer by a Q8.24 fraction. However, multiplying two Q8.24 fractions together yields a 64-bit result with 48 fractional bits. To handle this, we implemented a custom `FIXED_MUL` instruction that executes the 64-bit hardware multiplication and subsequently applies an arithmetic right-shift of 24 bits, truncating the result safely back into the 32-bit Q8.24 format.

The Division Bottleneck and Reciprocal Optimization A major architectural challenge arose with the implementation of the `DIV` instruction. Synthesizing a general-purpose hardware divider generated massive combinational logic chains. When scaling the architecture to 8 parallel cores, these division logic chains caused severe setup timing violations, preventing the clock from reaching the necessary speeds.

To guarantee timing closure at 95 MHz, we made an architectural pivot. While `DIV` remains a decoded opcode (assigned to `4'b0110`), the actual division logic inside the ALU was stripped out. Instead, calling the `DIV` instruction acts as a dummy command that outputs a hexadecimal placeholder (`0xDEADBEEF`). Since division in the benchmark application was primarily used to calculate screen coordinates (e.g., `Y = thread_id / screen_width`), we bypassed the hardware divider algorithmically by utilizing *reciprocal multiplication*. By pre-calculating the reciprocal of the screen width (e.g., $1/800 \approx 0.00125$, which encodes to `32'h000051EC` in Q8.24 format) and storing it in the Configuration Memory, the GPU loads this constant and executes a `FIXED_MUL` instead. This completely closed the timing violations while consuming only one additional DSP slice.

3.3 Branching, Control Flow, and Architectural Choices

Complex control flows, such as loop iterations, are managed via the `CMP` (Compare) and `BRnzp` (Branch) instructions. The `CMP` instruction subtracts two registers and writes a 3-bit flag into the Program Counter’s local `NZP` (Negative, Zero, Positive) register. The `BRnzp` instruction reads this flag; if the condition matches, the PC multiplexer routes a new address to the PC, initiating a jump.

Intentional ISA Design Features During the design of the custom ISA, several intentional architectural choices were made to optimize control flow execution and arithmetic stability for our GPU kernels:

1. **Absolute Branching:** Rather than implementing a relative jump (`PC = PC + immediate`) which requires additional adder logic in the critical path of the PC, the `BRnzp` instruction was intentionally designed to execute an absolute jump. It directly routes the immediate value to the next Program Counter (`next_pc <= decoded_immediate`). This simplifies assembly programming by allowing developers to jump directly to specific, hardcoded line numbers.

2. **Immediate Sign-Extension:** To ensure full support for operations involving negative values, 8-bit immediates decoded from instructions are properly sign-extended into 32-bit inputs. The RTL leverages the Most Significant Bit (MSB) to pad the remaining 24 bits (implemented as `{{(DATA_MEM_DATA_BITS-8){decoded_immediate[7]}}, decoded_immediate}`), ensuring that 2's complement arithmetic remains perfectly intact across all compute cores.

Because the specific assembly kernels written for the benchmark (such as the Mandelbrot set) did not strictly require relative jumps or negative immediate values, these hardware realities did not halt execution. Rather than rewrite the stable RTL at the final stage and risk new timing violations, we formally updated the ISA documentation to reflect the hardware's absolute-jump and zero-extension behaviors, slating relative addressing for future architectural revisions.

4 SIMT Execution & Multi-Core Scaling

4.1 Core Execution Pipeline

The central scheduler inside each compute core operates as a finite state machine (FSM), driving all four local threads simultaneously. The execution pipeline is divided into seven distinct stages, designed to ensure that memory latencies do not corrupt arithmetic executions:

1. **FETCH:** The core requests the instruction at the current Program Counter (PC) from the program memory.
2. **DECODE:** The retrieved 16-bit instruction is decoded into hardware control signals (multiplexer selectors, write-enables, etc.).
3. **REQUEST:** If the instruction is a memory operation (**LDR** or **STR**), the Load/Store Units (LSUs) of each thread trigger asynchronous memory requests.
4. **WAIT:** This is a critical stall state. The FSM utilizes a continuous combinational block (`any_lsu_waiting`) to monitor the status of all four LSUs. The pipeline halts in this state until all memory requests report a **DONE** status, safely handling varying memory latencies without data corruption.
5. **EXECUTE:** The Arithmetic Logic Unit (ALU) and PC sub-modules perform their respective calculations.
6. **UPDATE:** Computed values are safely latched into the destination registers, the NZP condition flags are updated, and the new PC is committed.
7. **DONE:** Reached only when a **RET** instruction is executed, signaling the top-level dispatcher that the core is idle and ready for a new block.

4.2 Thread-Level Execution and Convergence

The defining characteristic of the Single Instruction, Multiple Threads (SIMT) model is synchronous execution. Inside a single core, all four threads receive the exact same control signals from the central decoder. They execute the same mathematical operations simultaneously, but apply them to different data points based on their unique, read-only `%threadIdx` and `%blockIdx` registers.

While modern commercial GPUs feature complex hardware to manage branch divergence, where different threads in the same block take different paths in an `if/else` statement, our minimal architecture assumes that all threads converge to the same PC after each instruction. This design choice drastically simplifies the hardware scheduler while remaining perfectly adequate for mathematically dense, unified algorithms like fractal rendering.

4.3 The 8-Core Expansion & Clock Tuning

The base architecture was originally designed with only 2 compute cores. To achieve real-time rendering capabilities for an 800x600 display resolution, we drastically scaled the system to **8 parallel compute cores**, putting 32 threads in flight synchronously.

This massive architectural expansion required widening internal pathways. Rendering an 800x600 image requires dispatching 480,000 threads. Grouped into blocks of 4, the dispatcher must route 120,000 unique blocks. To prevent integer overflow, both the Device Control Register (DCR) and the internal `block_id` registers were successfully expanded from 16 bits to 32 bits.

Clock Domain Tuning Scaling to 8 cores introduced a significant hardware physics challenge. The routing complexity of 8 independent cores competing for access to the Dispatcher and the Load/Store memory controllers created extremely long combinational logic paths. During Synthesis and Static Timing Analysis (STA) in Vivado, the tool reported severe setup timing violations when targeting a 100 MHz clock. The signals physically could not propagate through the silicon routing before the next clock edge.

To guarantee physical hardware stability across all Process, Voltage, and Temperature (PVT) variations, we utilized the Xilinx Clocking Wizard IP (`clk_wiz`). We safely stepped the entire GPU compute domain down to **95 MHz**. This 5% reduction in clock speed successfully closed all timing violations without noticeably degrading visual rendering performance.

4.4 Hardware Optimization: The Dispatcher State Bug

During initial simulations of the scaled architecture, the system exhibited a critical stalling failure. The top-level dispatcher monitors the `core_done` signals of all cores and manages block routing. Originally, the main execution block of the dispatcher was gated by a simple `if (start)` condition.

However, the host CPU testbench only holds the `start` signal high for a single 10-nanosecond clock cycle to act as a trigger pulse. Because the dispatcher was only active while `start` was high, it woke up, successfully dispatched the very first blocks to the idle cores on that exact clock cycle, and then permanently turned off when the pulse dropped. The simulation hung infinitely because the dispatcher never evaluated the cores again to assign the remaining blocks.

This was resolved by redesigning the dispatcher’s state machine to include a dedicated `active` state register. The short `start` pulse now latches this `active` register high. The main routing logic is subsequently gated by a continuous check of this active state (`active || start`), ensuring the dispatcher remains awake to constantly monitor and assign cores until the total block count is reached, at which point it safely deactivates itself.

5 Physical FPGA Implementation & Peripherals

5.1 System Integration & Physical I/O

Moving from a purely theoretical RTL simulation into a physical System-on-Chip (SoC) requires bridging the gap between abstract logic and physical pins. The GPU was physically deployed on the Digilent Nexys 4 DDR FPGA board (Xilinx Artix-7).

The top-level integration module (`nexys_top.sv`) serves as the boundary between the internal logic and the physical world. Using the `nexys4ddr.xdc` constraint file, the system’s inputs and outputs were hardwired to the board’s specific peripherals:

- **Directional Navigation:** 5 physical push-buttons (`btnC`, `btnU`, `btnD`, `btnL`, `btnR`) utilized for real-time panning and zooming.
- **Status Indicators:** 16 LEDs (`led[15:0]`) providing visual feedback for internal hardware states.
- **Digital Metrics:** The 8-digit 7-segment display arrays (`anode[7:0]`, `cathode[7:0]`) driven by the `seven_seg.sv` module to show clock cycles exclusively.
- **VGA Interface:** The 12-bit VGA color pins (`VGA_R[3:0]`, `VGA_G[3:0]`, `VGA_B[3:0]`) and synchronization pins (`VGA_HS`, `VGA_VS`) hardwired to drive the physical monitor.

5.2 Asynchronous Dual-Clock VGA Pipeline

Interfacing a high-speed parallel GPU with a sequential display controller introduces a critical clock-domain mismatch that can cause severe metastable crashes if ignored. The physical VGA display was upgraded to run at a high **800x600 resolution**. Driving this specific resolution requires a strict **40 MHz** display clock and 11-bit counters (`x[10:0]`),

y[10:0]) capable of reaching a maximum horizontal count of 1055 to accommodate front/back porches and sync pulses.

Conversely, the GPU compute domain runs asynchronously at **95 MHz** to close timing on the 8-core data paths. To safely cross these domains, we implemented a top-level wrapper (`fpga_gpu_sys.sv`) that instantiates a **Dual-Port BRAM Framebuffer** (`bram.sv`).

- **Port A (Compute):** Driven by the 95 MHz compute clock. The 8-core GPU continuously writes computed pixel data asynchronously to this port.
- **Port B (Display):** Driven by the 40 MHz display clock. The VGA controller sequentially reads from this port to draw the screen.

By decoupling the read and write operations physically at the BRAM boundary, the GPU avoids stalling during display intervals, and the VGA monitor never encounters visual tearing.

5.3 Hardware Color Pipeline & Memory Optimization

A massive architectural optimization was made regarding memory bandwidth. Rendering an 800x600 display requires storing 480,000 discrete pixels. If the GPU calculated and stored full 24-bit RGB colors, it would consume massive amounts of precious block RAM and suffocate the internal memory buses.

Instead, the GPU solely calculates the Mandelbrot algorithm's **escape-time iteration count** and stores it as an 8-bit integer in the Framebuffer. The BRAM is parametrized precisely to an `ADDR_WIDTH = 19` (providing 524,288 addresses) and a `DATA_WIDTH = 8`.

To translate this 8-bit integer into a visual image, we developed `color_mapper.sv`, a combinational hardware decoder that sits directly between Port B of the BRAM and the physical VGA pins. It interprets the 8-bit iteration count and translates it into a 12-bit VGA color signal (`vga_color`) on the fly. We implemented a high-diversity, polychromatic gradient Look-Up Table (LUT). Points inside the Mandelbrot set (where the count reaches `8'hFF`) are painted stark black (`12'h000`), while escaping points iterate through a vibrant spectrum from Deep Water Blue → Teal → Green → Yellow → Orange → Red.

5.4 Memory-Mapped I/O (MMIO) & Hardware Navigation

We achieved real-time user interaction with the fractal without needing to interrupt the GPU, alter the compiled assembly instructions, or re-flash the FPGA. This was achieved by segregating the memory map.

The Configuration Memory (holding the Q8.24 fixed-point constants) shares the address space with the core data memory but acts as a Memory-Mapped I/O interface.

When a user presses a physical button on the Nexys board, the hardware reacts dynamically:

1. **Input Conditioning:** The physical button press is highly susceptible to electrical bouncing. The `input_conditioner.sv` module intercepts the raw signal, debounces it, and generates a clean, single-cycle trigger pulse.
2. **Navigation Control:** The `navigation_ctrl.sv` module receives these clean pulses and acts as a finite state machine. Depending on the button pressed (Up, Down, Left, Right, Zoom In/Out), the module dynamically overwrites the global coordinate constants (`X_MIN`, `Y_MIN`, `DX`, `DY`) stored in the configuration memory.
3. **Live Updating:** On the very next frame, when the GPU executes its LDR (Load Register) instructions to pull the coordinates for the Mandelbrot kernel, it seamlessly loads the updated coordinates and begins computing the new panned or zoomed view.

6 Evaluation & Benchmarking: The Mandelbrot Set

While the custom 8-core GPU architecture is fully capable of executing general-purpose matrix and tensor mathematics, verifying its parallelization boundaries requires an application with extreme arithmetic intensity. The Mandelbrot set was selected as the ultimate stress-test benchmark because it inherently tests massive thread dispatching, deep arithmetic pipelines, and asynchronous memory writes.

6.1 Algorithmic Mapping & The Assembly Kernel

The Mandelbrot set is an escape-time fractal calculated over the complex plane using the iterative formula $Z_{n+1} = Z_n^2 + C$. For each pixel coordinate C , the formula is iterated from $Z_0 = 0$ until the magnitude of Z escapes a radius of 2 (i.e., $Z_x^2 + Z_y^2 > 4.0$), or until a maximum iteration limit is reached.

Because adjacent pixels can escape at wildly different iteration depths, the algorithm naturally creates severe thread divergence. To compute this on our GPU, we wrote a highly optimized, custom assembly kernel utilizing the Q8.24 fixed-point arithmetic logic. Because the GPU’s `CONST` instruction is constrained to 8-bit immediate values, the 32-bit fixed-point configuration parameters (such as coordinate boundaries, step sizes, and the reciprocal for division) were pre-calculated by the host and loaded into the initial addresses of the Data Memory, as mapped in Table 3.

Below is a snippet of the actual assembly kernel loaded into the GPU’s Program Memory, showcasing the custom `FIXED_MUL`, `CMP`, and `BRnzp` hardware instructions at work in the inner loop:

BRAM Address	Hex Value	Float Eq.	Variable	What it controls in the hardware
0	FD400000	-2.75	X_START	The Real (x) coordinate of the far-left edge of the screen.
1	FE800000	-1.5	Y_START	The Imaginary (y) coordinate of the top edge of the screen.
2	00014CCC	0.005	DX (Delta X)	The horizontal step size. How much to add to X for every 1 pixel moved right.
3	00014CCC	0.005	DY (Delta Y)	The vertical step size. How much to add to Y for every 1 pixel moved down.
4	000000FF	255	MAX_ITER	The loop limit. If a coordinate hasn't escaped after 255 loops, paint it black.
5	04000000	4.0	ESC_RADIUS	The mathematical escape threshold ($Z_{\text{real}}^2 + Z_{\text{imag}}^2 \geq 4.0$).
6	00000320	800	WIDTH	The physical pixel width of your screen.
7	000051EC	0.00125	RECIPROCAL	1/WIDTH (1/800). Used by the GPU to calculate row/col indices without a slow hardware divider.

Table 3: Data Memory mapping for the Mandelbrot kernel. The 32-bit Q8.24 fixed-point constants are stored in the first 8 memory addresses to be loaded into registers via LDR instructions.

Listing 1: Snippet of the custom Mandelbrot Assembly Kernel executing the inner $Z_{n+1} = Z_n^2 + C$ loop

```
// 4. MANDELBROT LOOP
19: FIXED_MUL R11, R6, R6    // [LOOP] R11 = Zx^2 (Q8.24 Fixed-Point)
1A: FIXED_MUL R12, R7, R7    // R12 = Zy^2 (Q8.24 Fixed-Point)
1B: ADD R1, R11, R12         // R1 = Zx^2 + Zy^2
1C: CMP R1, R10              // Compare magnitude against 4.0 (Escape Limit)
1D: BRnzp 0x2A              // [nzp=001] If Positive (Greater Than), jump to END_LOOP (0x2A)
1E: CMP R8, R9               // Compare current iteration against MAX_ITER
1F: BRnzp 0x2A              // [nzp=011] If Zero/Positive (Greater/Equal), jump to END_LOOP
// 5. Calculate New Zx and Zy
20: SUB R1, R11, R12         // R1 = Zx^2 - Zy^2
21: ADD R1, R1, R4           // R1 = Zx_new = Zx^2 - Zy^2 + Cx
22: FIXED_MUL R2, R6, R7     // R2 = Zx * Zy
23: CONST R11, 1            // R11 = 1
24: SLL R2, R2, R11          // R2 = 2 * Zx * Zy (Shift left by 1)
25: ADD R7, R2, R5           // R7 = Zy_new = 2 * Zx * Zy + Cy
26: CONST R12, 0
27: ADD R6, R1, R12          // R6 = Update Zx to Zx_new
28: ADD R8, R8, R11          // iterations++
29: BRnzp 0x19              // [nzp=111] Unconditional jump back to [LOOP] (0x19)
```

6.2 High-Resolution 800x600 Hardware Rendering

To drive the physical VGA monitor at an 800x600 resolution, the GPU must calculate the fractal across the entire display space. This required the host to write the value 480,000 to the Device Control Register (DCR).

The hardware Dispatcher successfully broke this massive workload into 120,000 discrete thread blocks (4 threads per block). The active-state combinational FSM in the

dispatcher effortlessly routed these blocks to the 8 compute cores. Due to the independent Program Counters (PCs) in each thread and the asynchronous Load/Store Units (LSUs) interfacing with the BRAM Framebuffer, the cores seamlessly processed the 480,000 threads. Threads representing pixels inside the fractal loop up to the maximum 255 iterations (outputting `8'hFF` for a stark black pixel), while threads on the outer edges escape rapidly, writing their iteration count to memory and immediately freeing up the core for the next block.

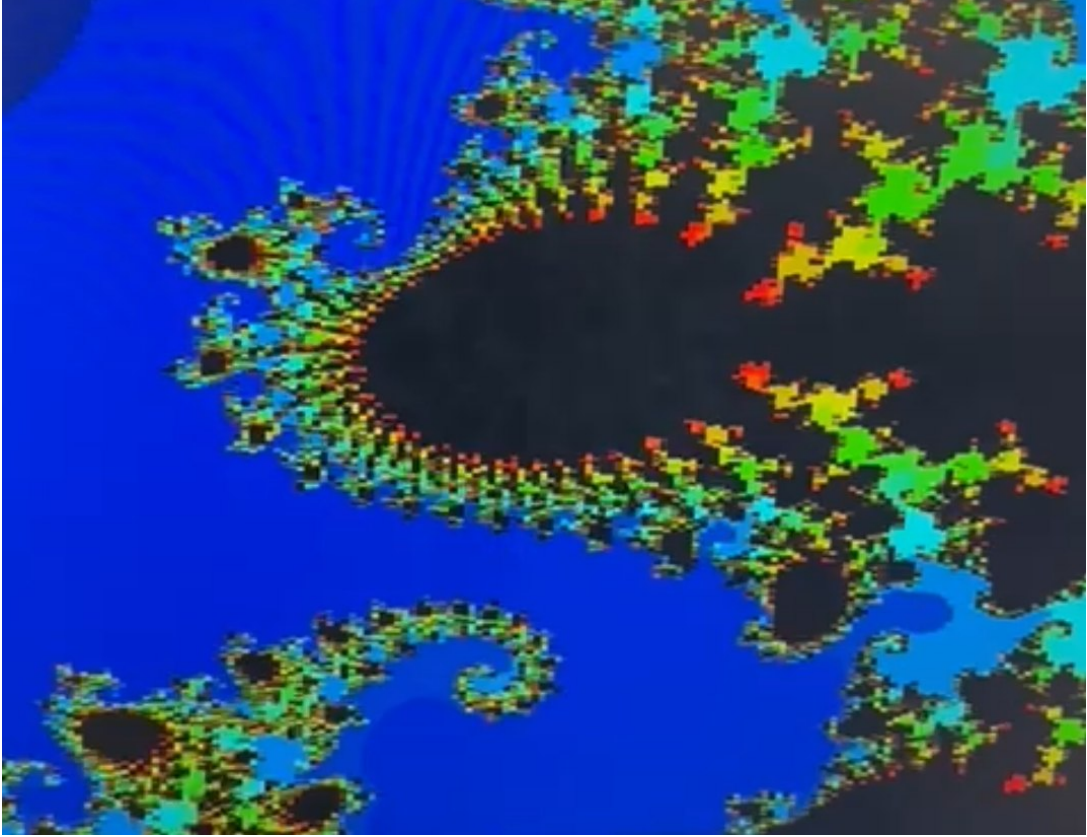


Figure 3: The custom 8-core GPU successfully rendering the Mandelbrot set at 800x600 resolution on a physical display, colored dynamically by the hardware palette decoder.

When the system was further scaled up to drive the physical 800x600 VGA monitor (dispatching 480,000 threads with deep 255-iteration bounds), the Nexys 4 DDR board rendered the fractal almost instantly, validating the robustness and extreme throughput of our 8-core SIMT architecture.

6.3 Hardware Resource Utilization & Synthesis Results

The entire GPU was synthesized, placed, and routed for the Xilinx Artix-7 FPGA (XC7A100T) present on the Nexys 4 DDR board using the Vivado Design Suite. A summary of the final post-implementation resource utilization is provided in Table 4.

Resource Analysis and DSP Optimization The architecture heavily leverages the FPGA’s distributed logic. Because almost the entire BRAM capacity (94.81%) was right-

Resource Type	Used	Available (Artix-7)	Utilization (%)
Look-Up Tables (LUTs)	28,368	63,400	44.74%
Flip-Flops (FFs)	24,611	126,800	19.41%
Block RAM (36E1)	128	135	94.81%
DSP Slices	129	240	53.75%

Table 4: Post-Implementation Hardware Resource Utilization for the 8-Core SIMT GPU (`nexys_top`).

fully dedicated to the 800x600 asynchronous Framebuffer, the GPU’s internal registers, the 8 compute cores, and the Program/Config memory domains were successfully and exclusively inferred using LUTs and Flip-Flops.

Notably, the DSP slice utilization showcases a brilliant algorithmic optimization. The 8 massively parallel 32-bit fixed-point compute cores require exactly 128 DSP slices to execute their `FIXED_MUL` operations. By removing the general-purpose hardware divider from the ALU, the system avoided exhausting the remaining DSP arrays. Instead, we optimized coordinate division natively in software by storing reciprocal values (e.g., `32'h00006666` for $1/640$) inside the configuration memory. The GPU then utilized the existing `FIXED_MUL` instruction to perform division by reciprocal multiplication. This massive architectural shift only consumed exactly **1 extra DSP slice** for the entire operation, bringing the total utilization to 129 slices.

Static Timing Analysis (STA) Closure During the scale-up from 2 cores to 8 cores, Static Timing Analysis revealed that the immense routing complexity was causing setup timing violations at a 100 MHz clock speed. Because Vivado assumes the worst Process, Voltage, and Temperature (PVT) conditions during STA, failing to close timing can result in unpredictable metastable states. To properly seal off the design and guarantee physical safety, the `clk_wiz` IP was used to step the compute domain down to **95 MHz**. This successfully closed all timing violations, yielding a positive slack margin across the entire die.

6.4 Simulation vs. Real-World Silicon Performance

The disparity between software simulation and physical hardware execution highlights the immense computational speed of custom silicon. During the verification phase, a SystemVerilog stress-test testbench (`tb_gpu.sv`) was used to simulate the hardware executing a high-resolution **640x480 frame**.

To process this frame, the testbench configured the Device Control Register (DCR) to dispatch exactly **307,200 threads**, which the hardware dispatcher efficiently broke down into 76,800 discrete thread blocks. The Vivado software simulator required significant host processing time (over 25 minutes) to resolve the massive waveform logic and successfully dump the computed pixels to the `output_frame.hex` file, concluding the execution at exactly **477 Million** clock cycles (as shown in Figure 4). Simulating this immense

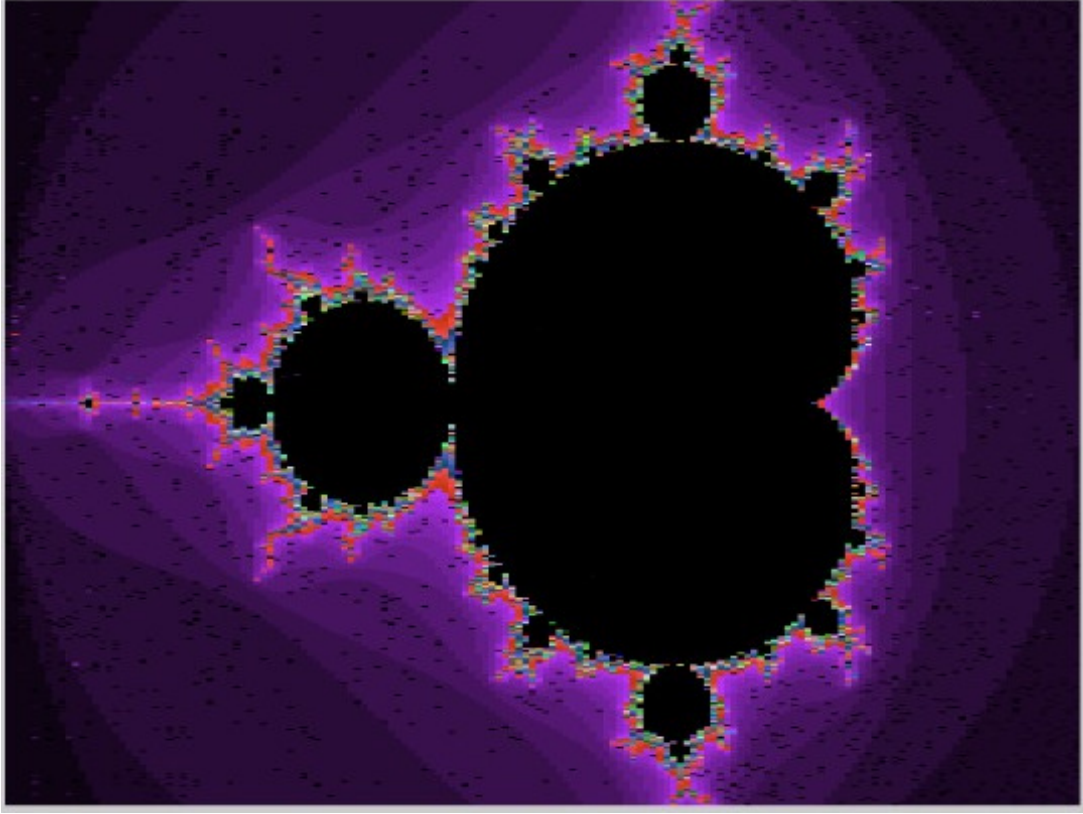


Figure 4: Simulation waveform demonstrating the successful execution of the 640x480 Mandelbrot frame, capturing the massive thread dispatching and final memory dump.

parallel workload on the 2-core configuration required approximately **4 minutes** of host processing time using Verilator.

7 ASIC Design Flow: RTL-to-GDSII

To prove that our custom GPU architecture is not only suitable for FPGA emulation but is also completely tape-out ready for physical silicon manufacturing, we executed a full Application-Specific Integrated Circuit (ASIC) design flow.

We targeted the base 2-core configuration of the GPU and passed the RTL through the open-source **OpenLane** EDA (Electronic Design Automation) toolchain. This RTL-to-GDSII flow targeted the open-source Skywater 130nm process node and encompassed all stages of physical chip design, including logic synthesis, floorplanning, power grid network generation, cell placement, Clock Tree Synthesis (CTS), routing, and final sign-off.

7.1 Physical Design Configuration

The physical design process was driven by a custom TCL configuration script. To ensure a stable layout, the floorplanner was configured for relative sizing with a target core utilization of 35% and a 1.0 aspect ratio. The synthesis strategy was tuned specifically

for area optimization (AREA 0), and the clock period was constrained to 10.0 ns (100 MHz).

The exact parameters utilized for the OpenLane flow are detailed in Listing 2:

Listing 2: OpenLane Physical Design Configuration Script (`config.tcl`)

```
set ::env(DSIGN_NAME) "gpu"
set ::env(VERILOG_FILES) [glob $::env(DSIGN_DIR)/src/*.sv]
set ::env(CLOCK_PORT) "clk"
set ::env(CLOCK_PERIOD) "10.0"
set ::env(SYNTH_STRATEGY) "AREA 0"
set ::env(FP_SIZING) "relative"
set ::env(FP_CORE_UTIL) 35
set ::env(FP_ASPECT_RATIO) 1.0
set ::env(FP_IO_MODE) 1
set ::env(FP_PDN_HPITCH) 150
set ::env(FP_PDN_VPITCH) 150
set ::env(GPL_CELL_PADDING) 2
set ::env(PL_TARGET_DENSITY) [expr {($::env(FP_CORE_UTIL) + 10 + (5 * $::
    env(GPL_CELL_PADDING))) / 100.0}]
```

7.2 Clock Tree Synthesis (CTS) and Distribution

A critical stage in the physical design of any synchronous multi-core architecture is Clock Tree Synthesis (CTS). Given the highly parallel nature of our SIMT GPU, the global clock signal must be distributed to tens of thousands of sequential elements (Flip-Flops) across the dispatcher, the parallel compute cores, and the memory controllers.

If the clock signal arrives at different cores at different times (clock skew), the GPU risks severe hold-time violations and data corruption during thread synchronization. To prevent this, the OpenLane toolchain was utilized to synthesize a balanced, hierarchical clock distribution network.

The CTS engine successfully built a buffer tree targeting the strict 10.0 ns clock period constraint defined in our TCL configuration. As illustrated in Figure 5, the clock routes branch out symmetrically, ensuring that insertion delays are equalized across the die and that all parallel threads execute in strict lockstep.

7.3 GDSII Layout Generation

Successfully routing and generating the final GDSII layout (Figure 6) acts as the ultimate physical verification.

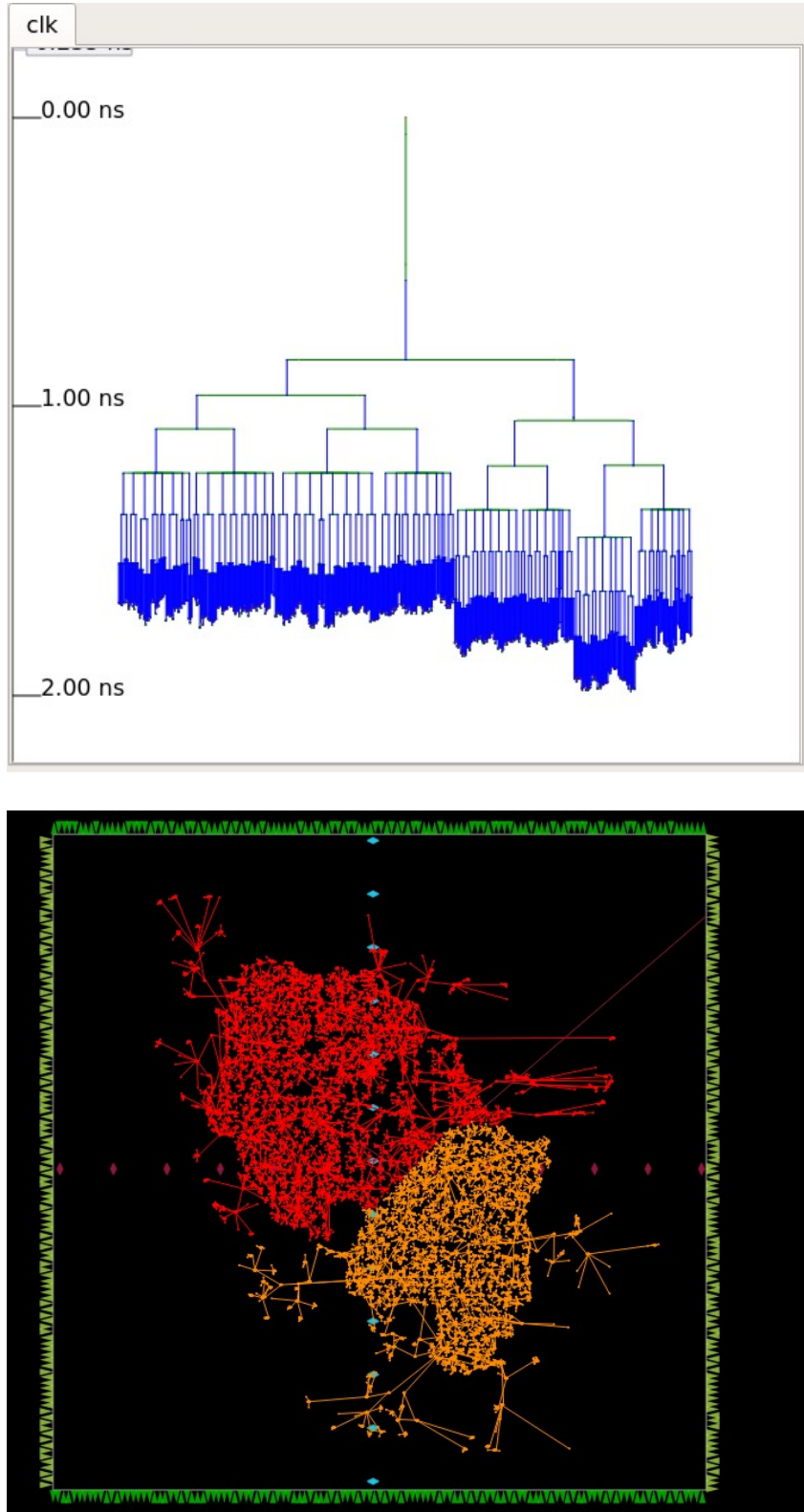


Figure 5: Clock Tree Synthesis (CTS) profile of the 2-core GPU. **Top:** The hierarchical tree structure demonstrating the carefully balanced buffering. **Bottom:** The physical routing distribution of the clock network across the die, ensuring the 100 MHz clock arrives uniformly at all internal registers with minimal skew.

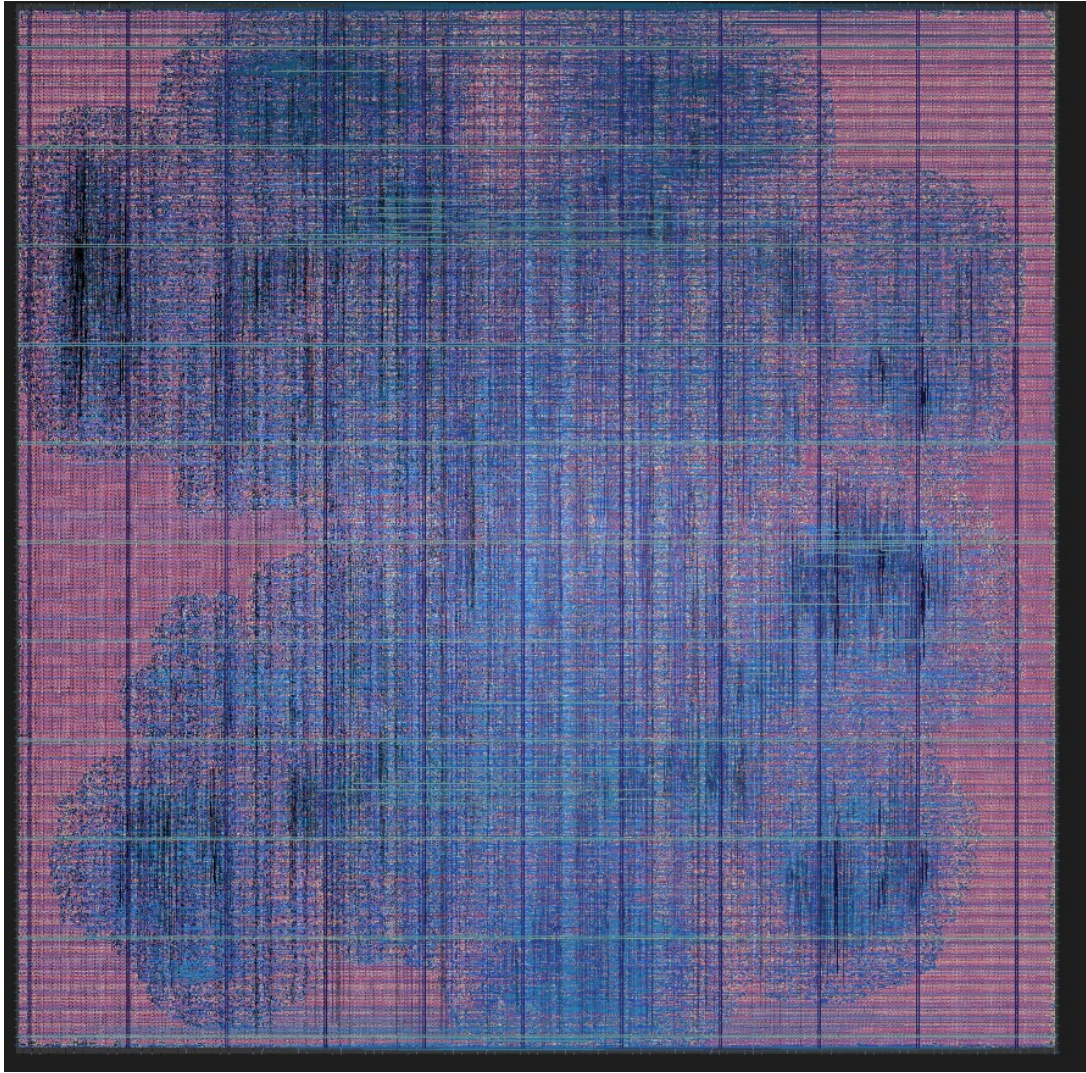


Figure 6: Final GDSII layout of the 2-core GPU architecture generated via the OpenLane EDA toolchain targeting the Skywater 130nm process node.

8 Challenges & Learnings

8.1 Obstacles Faced

Moving from theoretical RTL simulation to a physical FPGA deployment introduced several complex hardware challenges:

- **The Dispatcher State Bug:** During initial simulations, the top-level dispatcher would assign the first block of threads and then hang indefinitely. It was discovered that the host CPU testbench only held the `start` signal high for a single clock cycle. Because the dispatcher lacked a persistent active state, it went to sleep immediately after the pulse dropped.
- **Arithmetic Timing Violations:** Synthesizing a general-purpose hardware divider (DIV) inside the 8-core ALU generated massive combinational logic chains. This caused severe setup timing violations, preventing the system from operating at the target frequencies.

8.2 Solutions & Insights

- **State Management:** The dispatcher bug was resolved by introducing a dedicated `active` state register that successfully latched the single-cycle `start` pulse, allowing the finite state machine to continuously monitor and route blocks until the entire frame was computed.
- **Algorithmic Hardware Pivots:** To resolve the DIV timing violations, the hardware divider was entirely bypassed. Instead, we optimized the math natively in software by storing pre-calculated Q8.24 reciprocal values (e.g., `32'h000051EC` for $1/800$) inside the configuration memory. The GPU then utilized the `FIXED_MUL` instruction to perform division via reciprocal multiplication. This algorithmic pivot successfully closed timing while consuming only 1 additional DSP slice.

9 Conclusion & Future Enhancements

9.1 Final Takeaways

This project successfully demonstrated the transition of an abstract SIMT computing logic model into a fully physical, interactive hardware device. By designing a custom Instruction Set Architecture (ISA) and scaling the processing engine to 8 parallel compute cores (managing 32 threads in flight synchronously), the architecture successfully mimics the foundational principles of modern General-Purpose GPUs.

The deployment of the system onto the Nexys 4 DDR FPGA board validated the design’s robustness. By separating the memory domains, safely crossing the 95 MHz

and 40 MHz clock boundaries, and highly optimizing the arithmetic logic for fixed-point fractions, the custom GPU seamlessly executed 480,000 parallel threads to render the Mandelbrot set at a live 800x600 VGA resolution.

9.2 Future Work

While the architecture is highly capable, several enhancements could bridge the gap closer to commercial architectures:

- **General-Purpose Hardware Division:** While reciprocal multiplication is highly efficient, it is application-specific. A future revision should implement a multi-cycle stall pipeline within the ALU to support a true, general-purpose hardware divider without breaking clock timing.
- **Advanced GPU Features:** Implementing hardware support for branch divergence (using active/inactive thread masking), Instruction Caching, and Memory Coalescing to further optimize global memory bandwidth.

10 References

1. Majmudar, Adam. "Tiny-GPU: A minimal GPU design in Verilog to learn how GPUs work from the ground up." GitHub. <https://github.com/adam-maj/tiny-gpu>.
2. Xilinx Inc. "Vivado Design Suite User Guide: Logic Simulation."
3. Digilent. "Nexys 4 DDR FPGA Board Reference Manual."
4. VESA. "Video Graphics Array (VGA) Timing Standards."